Code Tracing and Trace Tables

Lecture 7

CS 1001

Kirk Duran

Today

- How to reason about a Python program as a sequence of state changes.
- How to build a trace table using **Line #** and **Known bindings**.
- How to distinguish program **state** from program **output**.
- How assignment, reassignment, arithmetic, strings, conversions, comparisons, and Boolean operators interact during execution.
- How to stop a trace at the first statement that cannot execute successfully.

Key idea

Tracing is not a new Python feature. It is a disciplined method for explaining what already-known Python statements do, one statement at a time.

Scope: Straight-Line Programs Only

Included

- assignment and reassignment,
- arithmetic expressions,
- `int`, `float`, `str`, and `bool`,
- type conversions,
- comparisons,
- `and`, `or`, and `not`,
- `print()`.

Not yet

- conditionals,
- loops,
- functions,
- lists,
- user input,
- exception handling,
- repeated or nested control flow.

Key idea

Every program in this lecture executes exactly once from top to bottom.

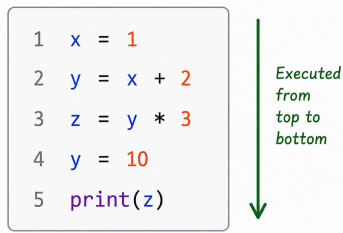
Part 1: Programs as Sequences of State Changes

Execution Has an Order

- Python executes a straight-line program in textual order: first statement, then second statement, then third statement, and so on.
- Each statement sees the program state produced by all successfully completed statements above it.
- A later assignment can change the state available to statements below it.
- A later assignment does *not* travel backward and recompute earlier assignments.

Execution order

`statement 1` → `statement 2` → `statement 3` → ⋯



Program State Means the Current Variable Associations

Definition

The **program state** is the collection of variable names and the values currently associated with those names at a particular moment during execution.

After one statement

```
x = 5
```

State: $\texttt{x} \rightarrow 5$

After another statement

```
y = x + 2
```

State: $\texttt{x} \rightarrow 5, \quad \texttt{y} \rightarrow 7$

Key idea

A trace table is a history of these states.

Trace One Assignment

Program

```
x = 5
```

Line #	Known bindings
1	*none*
End	`x -> 5`

1. Evaluate the right-hand side: `5` is already a value.
2. Associate `x` with `5`.
3. Advance to the next line; the new binding is visible in the next row of the trace.

Dependent Assignments Use the Current State

Program

```
x = 5
y = x * 2
```

Line #	Known bindings
1	*none*
2	`x -> 5`
End	`x -> 5, y -> 10`

Key idea

When the second statement executes, `x` already has a current value. Python retrieves that value before evaluating `x * 2`.

Activity: Predict the State Before We Trace

Program

```
base = 8
height = base + 4
area = base * height
```

In class

Without running Python, determine the bindings known when execution reaches each line. Include an **End** row to show the state after the final statement.

Questions to ask yourself

What values exist before each statement? Which names are read? Which name, if any, is updated?

Part 2: A Disciplined Tracing Procedure

The Five-Step Tracing Procedure

1. Record the current line number and all bindings known **before** that line executes.
2. Evaluate the statement using only those currently known bindings.
3. If the statement is an assignment, update the set of bindings.
4. If the statement is `print()`, record the displayed output without changing the bindings.
5. Advance to the next line and record the updated bindings there. After the final line, record an **End** row.

Key idea

A numbered row describes the state *when execution reaches that line*. The effects of that line appear in the next row.

A Trace Table Separates Time, State, and Output

- The **Line #** column identifies the statement Python is about to execute.
- **Known bindings** records every variable-value association already established when execution reaches that line.
- Therefore, line 1 begins with no known program-variable bindings.
- The effect of an assignment first appears on the *next* row; an **End** row records the state after the final statement.
- Program output is recorded separately so that bindings and displayed output are not conflated.

Line #	Known Bindings	Output
1		
2		
3		
4		
5		
End		

Worked Trace: Assignment, Dependency, Reassignment, Output

Program

```
x = 5
y = x * 2
x = y - 3
print(x)
```

Line #	Known bindings
1	*none*
2	`x -> 5`
3	`x -> 5, y -> 10`
4	`x -> 7, y -> 10`
End	`x -> 7, y -> 10`

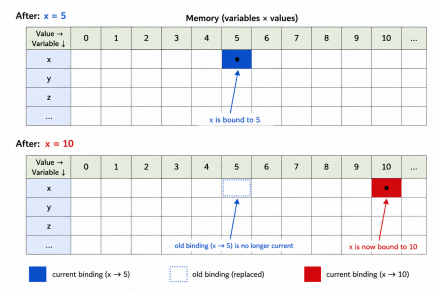
Output produced at line 4: `7`

Reassignment Replaces a Variable's Current Association

Program fragment

```
x = 5
x = 10
```

- After the first statement, x refers to 5.
- After the second statement, x refers to 10.
- The old association is no longer the current state for x.



Reassignment Does Not Recompute Earlier Assignments

Program

```
x = 3
y = x + 2
x = 10
```

Line #	Known bindings
1	*none*
2	`x -> 3`
3	`x -> 3, y -> 5`
End	`x -> 10, y -> 5`

Key idea

Assignment stores the value computed at that moment. `y` does not contain a live formula that automatically follows future changes to `x`.

Activity: Trace Reassignment Carefully

Program

```
a = 4
b = a + 6
a = b * 2
c = a - b
```

In class

Build the trace table using **Line #** and **Known bindings**. Circle the row where the binding for `a` changes from its earlier value.

Part 3: State Is Not Output

Assignment Changes State; `print()` Produces Output

Assignment

```
x = 7
```

- Updates program state.
- Displays nothing by itself.

Output

```
print(x)
```

- Reads the current value of `x`.
- Displays that value.
- Does not reassign `x`.

Trace State and Record Output Separately

Program

```
price = 12
quantity = 3
total = price * quantity
print(total)
```

Line #	Known bindings
1	*none*
2	`price -> 12`
3	`price -> 12, quantity -> 3`
4	`price -> 12, quantity -> 3, total -> 36`
End	`price -> 12, quantity -> 3, total -> 36`

Output produced at line 4: `36`

Activity: Predict Both State and Output

Program

```
minutes = 90
hours = minutes / 60
print(hours)
minutes = 120
```

In class

Trace the program. After the final statement, what is the state? What was displayed earlier? Keep those answers separate.

Part 4: Tracing Arithmetic and Basic Types

Arithmetic Expressions Are Evaluated Before Assignment

Program

```
width = 8
height = 5
area = width * height
perimeter = 2 * (width + height)
```

Evaluation of one statement

```
perimeter = 2 * (width + height)
```

↓ substitute current values

```
perimeter = 2 * (8 + 5)
```
→ `perimeter = 26`

Code Writing: Rectangle Calculator

In class

Write a straight-line Python script for a rectangle with width `7.5` and height `4`.
Your script should:

1. store the width and height,
2. calculate the area,
3. calculate the perimeter,
4. print the area,
5. print the perimeter.

Key idea

Before running it, predict the value and type of every variable.

One Possible Rectangle Calculator

Script

```
width = 7.5
height = 4
area = width * height
perimeter = 2 * (width + height)
print(area)
print(perimeter)
```

- `width` is a `float`; `height` is an `int`.
- The arithmetic expressions involving `width` produce floating-point results.
- The two `print()` statements produce output but do not modify the four variables.

Division Often Introduces a Floating-Point Value

Program

```
distance = 150
time = 2
speed = distance / time
```

Line #	Known bindings
1	*none*
2	`distance -> 150`
3	`distance -> 150, time -> 2`
End	`distance -> 150, time -> 2, speed -> 75.0`

Key idea

A correct trace records both value and, when relevant, type: `75.0` is not written as 75.

Strings Can Depend on Numeric Computations

Program

```
units = 4
hours = 3.5
workload = units * hours
label = "Estimated workload:  " + str(workload)
```

Last statement

`workload` has the numeric value `14.0`.

`str(workload)` produces the string `"14.0"`.

Concatenation produces `"Estimated workload:  14.0"`.

Trace Mixed Numeric and String Values

Line #	Known bindings
1	*none*
2	`units -> 4`
3	`units -> 4, hours -> 3.5`
4	`units -> 4, hours -> 3.5, workload -> 14.0`
End	`units -> 4, hours -> 3.5, workload -> 14.0, label -> "Estimated workload:  14.0"`

Key idea

The trace records the value actually stored in each variable, not merely the source expression that produced it.

Code Writing: Workload Estimate

In class

Write a straight-line script that stores:

- `courses = 5`,
- `hours_per_course = 4.5`,
- the total weekly hours,
- a string such as `"Weekly hours:  22.5"`.

Print the final string. Use `str()` only where it is needed.

One Possible Workload Script

Script

```
courses = 5
hours_per_course = 4.5
weekly_hours = courses * hours_per_course
summary = "Weekly hours:  " + str(weekly_hours)
print(summary)
```

- The arithmetic result is `22.5`.
- The conversion produces `"22.5"` for concatenation.
- The variable `weekly_hours` remains numeric; converting it for the string does not reassign it.

Part 5: Tracing Comparisons and Boolean Values

Comparisons Produce Values That Can Be Traced

Program

```
score = 84
minimum_score = 70
is_passing = score >= minimum_score
```

Evaluation

`score >= minimum_score` $\rightarrow$ `84 >= 70` $\rightarrow$ `True`

Key idea

The comparison is evaluated first; assignment then associates `is_passing` with the Boolean result.

Boolean Variables Become Part of Program State

Program

```
score = 84
minimum_score = 70
is_passing = score >= minimum_score
is_honors = score >= 90
eligible = is_passing and not is_honors
```

Line #	Known bindings
1	*none*
2	`score -> 84`
3	`score -> 84, minimum_score -> 70`
4	`score -> 84, minimum_score -> 70, is_passing -> True`
5	`score -> 84, minimum_score -> 70, is_passing -> True, is_honors -> False`
End	`score -> 84, minimum_score -> 70, is_passing -> True, is_honors -> False, eligible -> True`

Do Not Confuse Assignment with Equality Comparison

Assignment

```
x = 10
```

Changes the state of `x`.

Comparison

```
x == 10
```

Computes `True` or `False`; does not change `x`.

In class

Suppose `x` currently equals `7`. Describe the effect and result of each expression above.

Code Writing: A Simple Budget Check

In class

Write a straight-line script that stores:

- `budget = 100`,
- `item_price = 64.50`,
- `tax_rate = 0.08`,
- the tax,
- the total cost,
- a Boolean named `within_budget` that compares the total cost with the budget.

Print `total_cost` and `within_budget`. Do not use a conditional statement.

One Possible Budget Script

Script

```
budget = 100
item_price = 64.50
tax_rate = 0.08
tax = item_price * tax_rate
total_cost = item_price + tax
within_budget = total_cost <= budget
print(total_cost)
print(within_budget)
```

Key idea

A Boolean can summarize a relationship among previously computed values even though the program does not yet make a decision based on it.

Part 6: Full Trace Synthesis

Integrated Example

Program

```
x = 3
y = x + 2
x = 10
z = x + y
larger = z > 10
same = x == y
result = larger and not same
print(result)
```

In class

Before tracing, predict the final value and type of every variable.

Integrated Example: Complete Trace

Line #	Known bindings
1	*none*
2	`x -> 3`
3	`x -> 3, y -> 5`
4	`x -> 10, y -> 5`
5	`x -> 10, y -> 5, z -> 15`
6	`x -> 10, y -> 5, z -> 15, larger -> True`
7	`x -> 10, y -> 5, z -> 15, larger -> True, same -> False`
8	`x -> 10, y -> 5, z -> 15, larger -> True, same -> False, result -> True`
End	`x -> 10, y -> 5, z -> 15, larger -> True, same -> False, result -> True`

Output produced at line 8: `True`

Read the Integrated Trace as a Sequence of Questions

1. When `y = x + 2` executes, which `x` exists? **3**.
2. After `x = 10`, does `y` change? **No**; it remains **5**.
3. What values does `z = x + y` read? **10** and **5**.
4. What does `z > 10` compare? **15** and **10**.
5. What does `x == y` compare? **10** and **5**.
6. What does `larger and not same` evaluate to? **True**.
7. What changes on the final line? **Output only**.

Activity: Trace a Calculation Script

Program

```
miles = 180
gallons = 6
mpg = miles / gallons
target = 28
meets_target = mpg >= target
message = "MPG: " + str(mpg)
print(message)
print(meets_target)
```

In class

Construct the complete trace table using Line # and Known bindings. Include variable types in annotations beside your table.

Part 7: Tracing to the First Error

A Trace Stops When a Statement Cannot Complete

Program

```
count = 5
label = "Count:  " + count
double = count * 2
```

- The first statement executes successfully: `count` becomes `5`.
- The second statement attempts to concatenate a string and an integer directly.
- That statement does not successfully produce a value for `label`.
- Execution stops there; the later assignment to `double` is not traced as executed.

Tracing rule

Identify the first failing statement. Do not invent state changes from statements that were never reached.

Trace Only the Successfully Completed State

Line #	Known bindings
1	*none*
2	`count -> 5`

Line 2 fails. No binding for `label` is created; execution stops with `count -> 5`.

In class

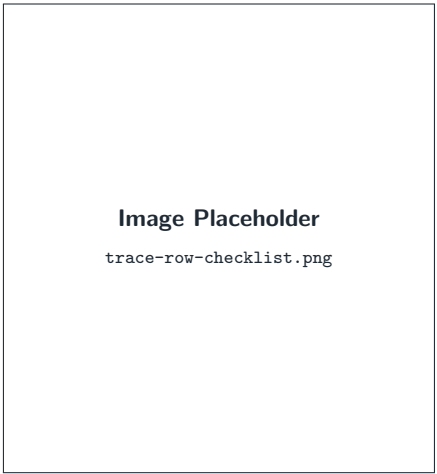
Rewrite only the failing statement so that the program can continue, using a type conversion already introduced in this course.

Common Tracing Errors

- Reading the whole program as though all assignments happen simultaneously.
- Forgetting that reassignment replaces a variable's current value.
- Recomputing a previous assignment after one of its source variables changes.
- Treating `print()` as though it changes program state.
- Confusing `=` with `==`.
- Treating `"5"` and `5` as interchangeable values.
- Skipping rows and losing the exact moment when state changes.
- Reporting only the final answer instead of showing the execution history.

A Useful Self-Check for Every Row

1. What statement just executed?
2. What values did it read?
3. What expression result did it compute?
4. What variable, if any, changed?
5. What output, if any, appeared?



Part 8: Writing Small Scripts You Can Trace

Code Writing: Temperature Conversion

In class

Write a straight-line script that:

1. stores `fahrenheit = 77`,
2. computes Celsius using `(fahrenheit - 32) * 5 / 9`,
3. builds a string beginning with `"Celsius:  "`,
4. prints the string,
5. stores a Boolean named `is_warm` that is true when Celsius is at least 20.

Predict the known bindings at each line before running it, and include the final bindings at **End**.

One Possible Temperature Script

Script

```
fahrenheit = 77
celsius = (fahrenheit - 32) * 5 / 9
label = "Celsius:  " + str(celsius)
print(label)
is_warm = celsius >= 20
```

- The calculation produces `25.0`.
- `label` stores `"Celsius:  25.0"`.
- Printing the label does not change it.
- `is_warm` becomes `True`.

Code Writing: Purchase Summary

In class

Write a script for a purchase of 3 notebooks at `2.75` dollars each.
Include variables for:

- quantity,
- unit price,
- subtotal,
- a fixed shipping cost of `4.00`,
- final total,
- a Boolean that records whether the final total is less than 15,
- a printable string summary of the final total.

From Writing to Tracing

When writing

- Choose meaningful variable names.
- Build calculations from previously available values.
- Keep each statement conceptually simple.
- Predict values before running.

When tracing

- Respect top-to-bottom order.
- Use only the current state.
- Record every state-changing statement.
- Separate state from displayed output.

Key idea

Writing and tracing reinforce the same execution model from opposite directions.